

WEEKLY ACTIVITY

1) Create a GitHub repository for the application

Ans : create the github repo

And clone in the ubuntu terminal

```
vrushali@Vrushali:~$ git clone https://github.com/Vrushk13/devops-jenkins-activity.git
Cloning into 'devops-jenkins-activity'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Create shell based application

App.sh

Run this application using the command : ./app.sh

```
vrushali@Vrushali:~/devops-jenkins-activity$ ./app.sh
Hello from Jenkins CI Pipeline
Jenkins Application build successful
vrushali@Vrushali:~/devops-jenkins-activity$
```

Create the Test file for the application

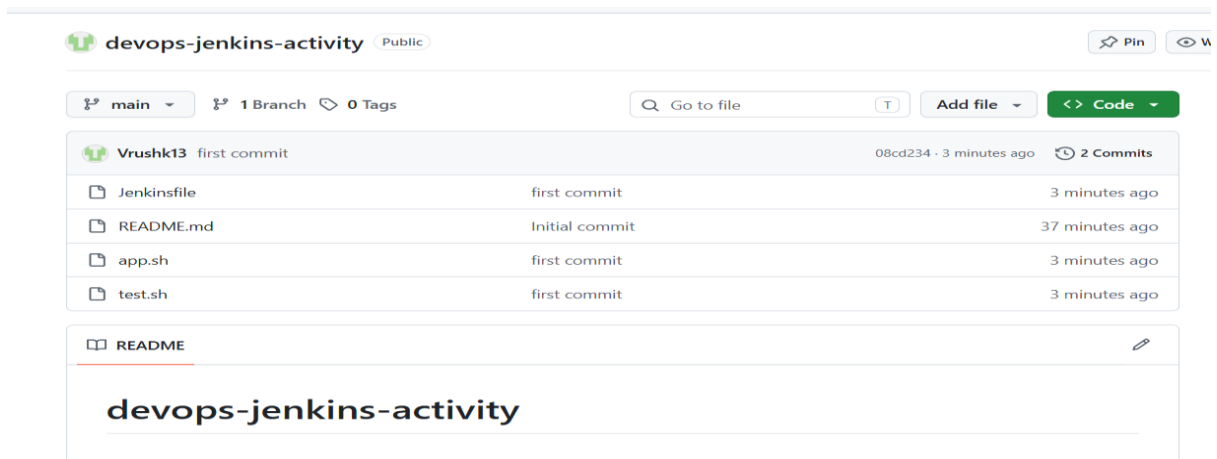
Nano test .sh

```
vrushali@Vrushali:~/devops-jenkins-activity$ nano test.sh
vrushali@Vrushali:~/devops-jenkins-activity$ ls
README.md  app.sh  test.sh
vrushali@Vrushali:~/devops-jenkins-activity$ chmod +x test.sh
vrushali@Vrushali:~/devops-jenkins-activity$ ./test.sh
Running application test...
TEST PASSED: app.sh exists.
vrushali@Vrushali:~/devops-jenkins-activity$
```

Create the Jenkins file to automate the process

Write the script

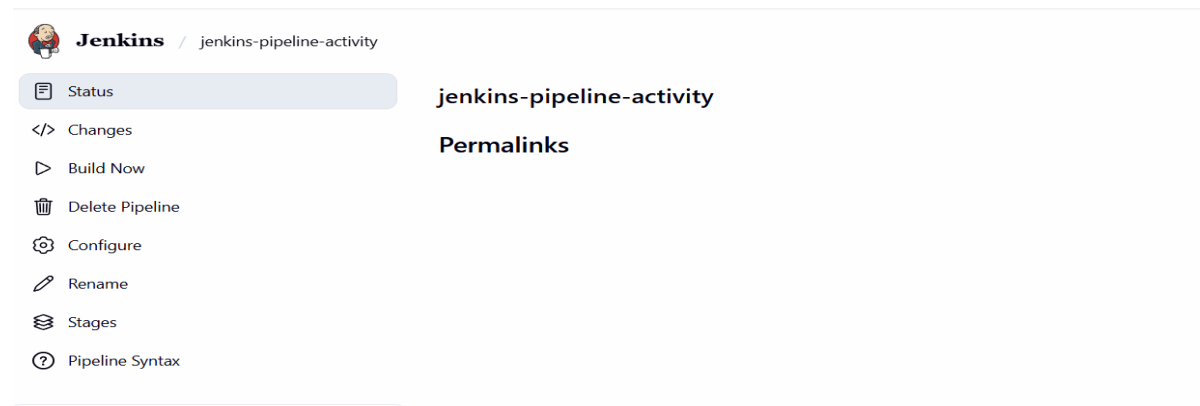
Push the code to the github



2) Connect github to Jenkins

Ans : Connect the github to Jenkins

Configure the github with Jenkins and create the pipeline



After the build the pipeline

```
> git config core.sparsecheckout # timeout=10
> git checkout -f 08cd23415da4b702c3ea26548e8f83b4e83fe06d # timeout=10
Commit message: "first commit"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building application...
[Pipeline] sh
+ chmod +x app.sh
[Pipeline] sh
+ ./app.sh
Hello from Jenkins CI Pipeline
Jenkins Application build successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running tests...
[Pipeline] sh
+ chmod +x test.sh
```

```

+ ./test.sh
Running application test...
TEST PASSED: app.sh exists.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Validation)
[Pipeline] echo
Validating application...
[Pipeline] sh
+ test -f app.sh
[Pipeline] sh
+ test -f test.sh
[Pipeline] echo
Validation successful!
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI Pipeline completed successfully!

[Pipeline] echo
CI Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

3) Configure Github Webhook

To configure github webhook first we have to build the pipeline successfully

And then configure the github hook

1) In Jenkins configure the github hook

Configure

General
Triggers
Pipeline
Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Build after other projects are built ?
☐ Build periodically ?
☒ GitHub hook trigger for GITScm polling ?
☐ Poll SCM ?
☐ Trigger builds remotely (e.g., from scripts) ?

General
Access
Collaborators
Moderation
Code, planning, and automation
Branches
Tags
Rulesets
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages

Webhooks / Manage webhook

Settings
Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *
http://192.168.1.10:8080/github-webhook/

Content type *
application/json

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.
☒ Enable SSL verification
☐ Disable (not recommended)

Install the ngrok for Jenkins reach to the private IP
Configure it with github

Vrushk13 / devops-jenkins-activity

Type to search

+

Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings

General
Access
Collaborators
Moderation
Code, planning, and automation
Branches
Tags
Rulesets
Actions
Webhooks
Copilot

Webhooks

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

☒ https://lent-elevating-riches.ngrok-f... (push)

Edit
Delete

Last delivery was successful.

Webhooks / Manage webhook

Settings
Recent Deliveries

42ddb6e2-a085-11f1-8260-3ea48dd320a9
ping
2026-08-25 18:32:44

Request
Response
200

Redeliver
Completed in 0.53 seconds.

Request URL: https://lent-elevating-riches.ngrok-free.dev/github-webhook/
Request method: POST
Accept: */*

After changes in the code

```

jenkinsfile README.md app.sh test.sh
vrushali@Vrushali:~/devops-jenkins-activity$ echo "webhook test" >> README.md
vrushali@Vrushali:~/devops-jenkins-activity$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

```

After changes in the code and push into the github

Pipeline is automatically trigger

Builds >

Filter

Today

✓ #2 1:07 pm

✓ #1 12:49 pm

Console

Started by GitHub push by Vrushk13

Obtained Jenkinsfile from git <https://github.com/Vrushk13/devops-jenkins-activity.git>

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/jenkins-pipeline-activity

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Declarative: Checkout SCM)

[Pipeline] checkout

Selected Git installation does not exist. Using Default

The recommended git tool is: NONE

using credential 4461e62a-9b89-46e4-9557-e74dc96fchd9

[Pipeline] { (Declarative: Post Actions)

[Pipeline] echo

CI Pipeline completed successfully!

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] //withEnv

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

Webhooks / Manage webhook

Settings

Recent Deliveries

✓

deb29c90-a085-11f1-99fc-3265e8c401fa

push

2026-08-25 18:37:06

...

✓

42ddb6e2-a085-11f1-8260-3ea48dd320a9

ping

2026-08-25 18:32:44

...

4) Introduce a controlled error into the project.

Ans: In the test.sh file add the line to fail the pipeline

```
if [ -f wrong-file.sh ]; then
```

And we got the output like

```
+ ./test.sh
Running application test...
TEST FAILED: app.sh does not exist.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Validation)
Stage "Validation" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI Pipeline failed!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```

5) Fix the error and run the pipeline again.

Ans : First , Identify the issue and resolve the issue how to fix the error

I fix the error and run the pipeline

Go to your terminal:

```
cd ~/devops-jenkins-activity
```

Open test.sh:

```
nano test.sh
```

Find:

```
if [ -f wrong-file.sh ]; then
```

Change it to:

```
if [ -f app.sh ]; then
```

Failure

Test stage failed.

TEST FAILED: app.sh does not exist.

ERROR: script returned exit code 1

Finished: FAILURE

Root cause

The test script was incorrectly checking for wrong-file.sh instead of app.sh.

Troubleshooting

1. Opened Jenkins failed build.
2. Opened Console Output.
3. Identified the failed Test stage.
4. Found exit code 1.
5. Checked test.sh.
6. Found incorrect file reference.

Resolution

Changed:

wrong-file.sh

to:

app.sh

Short troubleshooting summary

The Jenkins pipeline failed during the **Test** stage because test.sh was checking for the wrong file (wrong-file.sh). I checked the Jenkins Console Output, identified the error and exit code 1, corrected it to app.sh, and pushed the fix to GitHub. Jenkins then automatically triggered the pipeline again, and all stages completed successfully.